Setup guide and how it works

Current project guide | Julia 1.13 | October 2026

What it does

Lecture Learner turns a lecture transcript into Markdown study notes. It lists main points with supporting points and separates course announcements such as test dates, deadlines, and class changes.

The verified path reads a transcript and uses the pure-Julia summarizer. Optional cloud code can transcribe audio with OpenAI and request JSON notes from Claude or OpenAI; live cloud behavior has not been tested.

Run the verified offline sample

```
julia --project=. LectureLearner.jl --offline --text sample_transcript.txt
```

Run this from the project folder. It writes sample_transcript_notes.md beside the sample and prints the notes. No API key, network access, HTTP, or JSON3 is needed for this mode.

VERIFICATION STATUS
Offline transcript processing was manually tested and its sample notes were reviewed. The regression suite passed. Live cloud APIs and audio transcription were not tested.

Files to know

- LectureLearner.jl - command-line entry point, cloud requests, fallback, and output files.
- TextTools.jl - pure-Julia sentence, announcement, date, summary, and Markdown functions.
- Project.toml and Manifest.toml - declared and pinned cloud dependencies.
- sample_transcript.txt - sample input; test/runtests.jl - regression checks.
- .gitignore - excludes generated notes, recordings, models, and common secret files.

Setup and commands

Use the README for the shortest current instructions.

Julia environment

Use Julia 1.13. HTTP and JSON3 are the only direct external dependencies, declared in Project.toml and pinned in Manifest.toml. Install the pinned environment for cloud use and the full test suite:

```
julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

A fresh machine may need internet access to download packages. Offline transcript processing can run before installing them.

Choose a run mode

```
# Verified transcript-only path; no keys
julia --project=. LectureLearner.jl --offline --text sample_transcript.txt
# Optional cloud notes for an existing transcript
julia --project=. LectureLearner.jl --text sample_transcript.txt
# Optional audio transcription, then notes
julia --project=. LectureLearner.jl lecture.m4a
```

Set OPENAI_API_KEY in your shell for audio transcription. The default Claude notes backend uses ANTHROPIC_API_KEY. Never put key values in source files or presentation slides. Live API calls have not been verified.

Outputs and input checks

- Notes appear on screen and are saved beside the input as <name>_notes.md.
- Audio mode also saves <name>_transcript.txt. Existing files at those paths are replaced.
- Offline mode cannot transcribe audio; pass a nonempty file with --text.
- Missing files, empty transcripts, and invalid arguments produce an error and a nonzero exit.

How the pipeline works

The current implementation keeps one Markdown renderer for both paths.

1 Audio to transcript (optional cloud path)

transcribe_audio uploads the audio file to OpenAI using HTTP.Form and asks for plain text. It requires OPENAI_API_KEY. A blank transcription is rejected. This path is present in code but has not been tested with a live recording or API.

2 Transcript to notes

By default, llm_notes prompts Claude for JSON with a title, main_points (each with sub_points), and announcements (type, detail, date). The code also has an OpenAI notes backend selected by a constant. HTTP and JSON3 load only when a cloud path is used.

make_notes checks the parsed cloud response before rendering. A missing key, request failure, invalid JSON, or malformed note structure causes a warning and an offline fallback. The warning reports the error type, not the full exception.

3 Pure-Julia fallback

- Split the transcript into sentences, remove sign-offs, and find course announcements and dates.
- Use context to avoid treating technical mentions of tests, projects, or labs as announcements.
- Group remaining sentences by vocabulary similarity and spoken transitions.
- Rank sentences with TF-IDF; choose one main point and up to three supporting points per topic.

4 Markdown output and limits

TextTools.render formats announcements first, then numbered main points. The offline method extracts sentences rather than understanding the whole lecture, so it can omit details or group topics imperfectly. Check notes against the transcript before studying or presenting them.

Tests, troubleshooting, and demo

Use verified output for the presentation unless the team validates a live cloud run.

Regression tests

```
julia --project=. test/runtests.jl
```

The tests cover sample content, technical terms versus actual announcements, malformed cloud-response fallback, input errors, and offline startup without cloud packages. Cloud responses are mocked; a passing test suite does not prove live API access.

Common problems

- Cloud package cannot load: run Pkg.instantiate() with --project=. Offline --text still works without those packages.
- Missing OPENAI_API_KEY: audio transcription cannot start. Use --offline --text for an existing transcript.
- Notes say offline unexpectedly: cloud notes failed. The warning shows the error type; check the key, connection, provider response, and package setup.
- No output from an input: verify the file exists and contains nonempty transcript text.

Presentation plan

Put the tested offline command, its generated notes, and the run steps in the slides. That lets the team show the workflow without relying on a live request. If the team runs a live demo, skip the slide walkthrough. Ask Pao by email whether a live run is required.

DO NOT OVERCLAIM
Offline transcript processing was manually tested. Cloud APIs and audio transcription were not tested, so describe them as implemented paths, not verified results.